

INFORME PATRONES DE DISEÑO

Taller unidad 2 – Empresa Deccos

Felipe Saldías Miranda

ÍNDICE

Introducción.....	3
Actividad 1 – Menú de Navegación.....	4
Diagrama UML	4
Implementación del Código.....	4
Patrón Composite Aplicado.....	6
Actividad 2 – Integración Bodegaje–Catálogo Conclusiones	7
Diagrama UML	7
Implementación del Código.....	7
Patrón Adapter Aplicado	8
Actividad 3 – Usuario Anónimo	9
Diagrama UML	9
Implementación del Código.....	9
Patrón Null Object Aplicado	11
Composición de Patrones y Desacoplamiento del Sistema	11
Diagrama de Colaboración	11
Beneficios de la Composición	11
Conclusión	12

Introducción

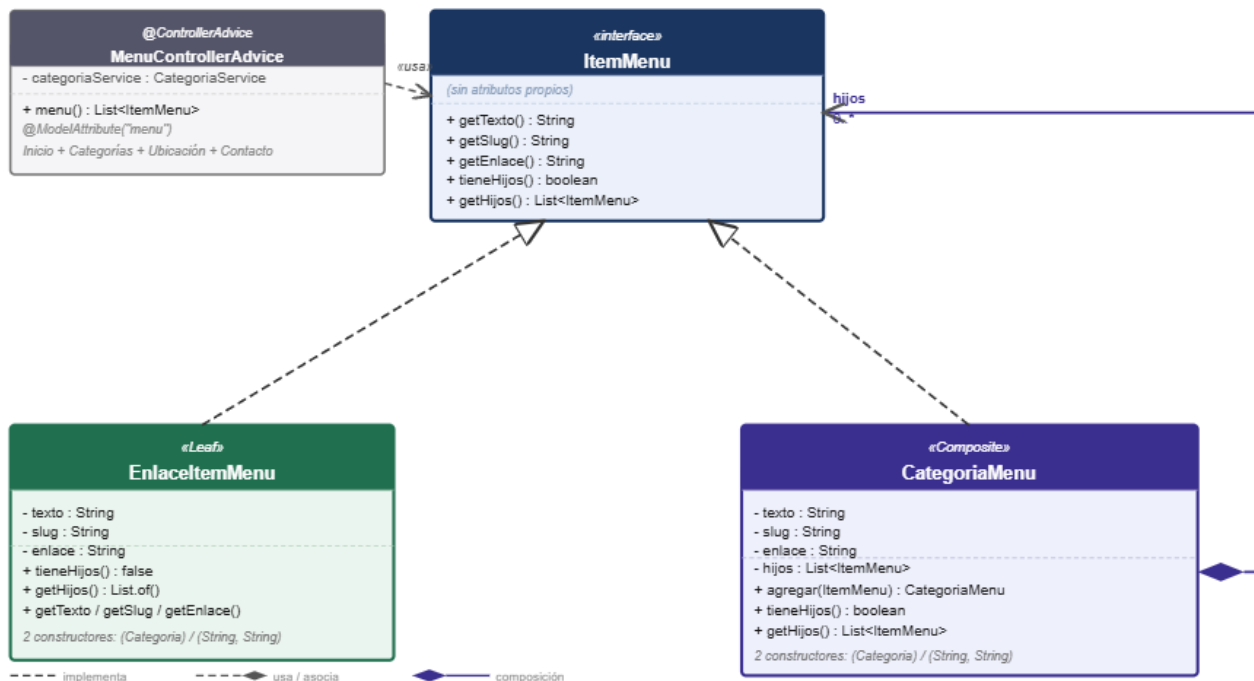
El presente informe documenta el diseño e implementación de tres soluciones basadas en patrones de diseño del catálogo GoF (Gang of Four) aplicadas al sistema de tienda en línea *Deccos*. El sistema está desarrollado con Spring Boot 3.4.2 sobre Java 17, utilizando Thymeleaf como motor de plantillas, Spring Security para autenticación y autorización, Spring Data JPA con una base de datos H2 en memoria.

Cada actividad presenta un problema de integración o robustez concreto, para el cual se selecciona, diseña y justifica el patrón de diseño más adecuado. Las soluciones no son independientes: los tres patrones colaboran en tiempo de ejecución para producir las páginas del sitio web.

El documento cubre los siguientes aspectos por cada actividad: identificación del problema, selección y justificación del patrón, interfaces involucradas, diagrama de clases UML, e implementación completa del código.

Actividad 1 – Menú de Navegación

Diagrama UML



Implementación del Código

Interfaz ya provista por el enunciado. Define el contrato único que la plantilla Thymeleaf consume, independiente del tipo concreto del nodo.

cl/patrones/taller/u2/tienda/menu/ItemMenu.java

```

public interface ItemMenu {
    public String getTexto();
    public String getSlug();
    public String getEnlace();
    public boolean tieneHijos();
    public List<? extends ItemMenu> getHijos();
}
  
```

Representa un enlace estático o una subcategoría sin hijos propios. `tieneHijos()` retorna siempre false y `getHijos()` devuelve una lista inmutable vacía. Dispone de dos constructores: uno que recibe un objeto `Categoria` (para construir el enlace dinámicamente) y otro que acepta texto y URL directos (para los ítems estáticos).

cl/patrones/taller/u2/tienda/menu/EnlaceItemMenu.java

```

public class EnlaceItemMenu implements ItemMenu {

    private final String texto, slug, enlace;

    public EnlaceItemMenu(Categoria categoria) {
        this.texto = categoria.getNombre();
        this.slug = Slugger.toSlug(categoria.getNombre());
        this.enlace = "/" + categoria.getId() + "/" + this.slug;
    }

    public EnlaceItemMenu(String texto, String enlace) {
        this.texto = texto;
        this.slug = Slugger.toSlug(texto);
        this.enlace = enlace;
    }
}
  
```

```

@Override public boolean tieneHijos()           { return false; }
@Override public List<ItemMenu> getHijos()       { return List.of(); }
}

```

Representa un nodo con hijos. Almacena internamente una `List<ItemMenu>` que puede contener tanto hojas como otros compuestos, habilitando la recursividad. El método `agregar()` retorna `this` para permitir encadenamiento fluido al construir el árbol.

cl/patrones/taller/u2/tienda/menu/CategoriaMenu.java

```

public class CategoriaMenu implements ItemMenu {

    private final String      texto, slug, enlace;
    private final List<ItemMenu> hijos = new ArrayList<>();

    public CategoriaMenu(Categoria categoria) {
        this.texto = categoria.getNombre();
        this.slug  = Slugger.toSlug(categoria.getNombre());
        this.enlace = "/categoria/" + categoria.getId() + "/" + this.slug;
    }

    public CategoriaMenu(String texto, String enlace) {
        this.texto = texto;
        this.slug  = Slugger.toSlug(texto);
        this.enlace = enlace;
    }

    public CategoriaMenu agregar(ItemMenu item) {
        hijos.add(item);
        return this;
    }

    @Override public boolean tieneHijos()           { return !hijos.isEmpty(); }
    @Override public List<ItemMenu> getHijos()       { return hijos; }
}

```

La anotación `@ControllerAdvice` convierte esta clase en un interceptor global: el método anotado con `@ModelAttribute("menu")` es invocado antes de cada request y el valor retornado se inyecta automáticamente en el modelo Thymeleaf.

El algoritmo recorre la lista plana de categorías en dos pasadas: (1) identifica las raíces (sin padre) y construye un `CategoriaMenu` por cada una; (2) agrega como hojas (`EnlaceItemMenu`) las categorías cuyo padre coincide con la raíz en curso.

cl/patrones/taller/u2/tienda/controller/MenuControllerAdvice.java

```

@ControllerAdvice
public class MenuControllerAdvice {

    private final CategoriaService categoriaService;

    public MenuControllerAdvice(CategoriaService categoriaService) {
        this.categoriaService = categoriaService;
    }

    @ModelAttribute("menu")
    public List<ItemMenu> menu() {
        List<Categoria> todas = categoriaService.getCategorias();
        CategoriaMenu categoriasMenu = new CategoriaMenu("Categorías", "#");

        for (Categoria raiz : todas) {

```

```

        if (raiz.getPadre() != null) continue;

        CategoriaMenu raizMenu = new CategoriaMenu(raiz);

        todas.stream()
            .filter(c -> c.getPadre() != null
                && c.getPadre().getId().equals(raiz.getId()))
            .map(EnlaceItemMenu::new)
            .forEach(raizMenu::agregar);

        categoriasMenu.agregar(raizMenu);
    }

    List<ItemMenu> menu = new ArrayList<>();
    menu.add(new EnlaceItemMenu("Inicio", "/"));
    menu.add(categoriasMenu);
    menu.add(new EnlaceItemMenu("Ubicación", "/ubicacion"));
    menu.add(new EnlaceItemMenu("Contacto", "/contacto"));
    return menu;
}
}

```

Patrón Composite Aplicado

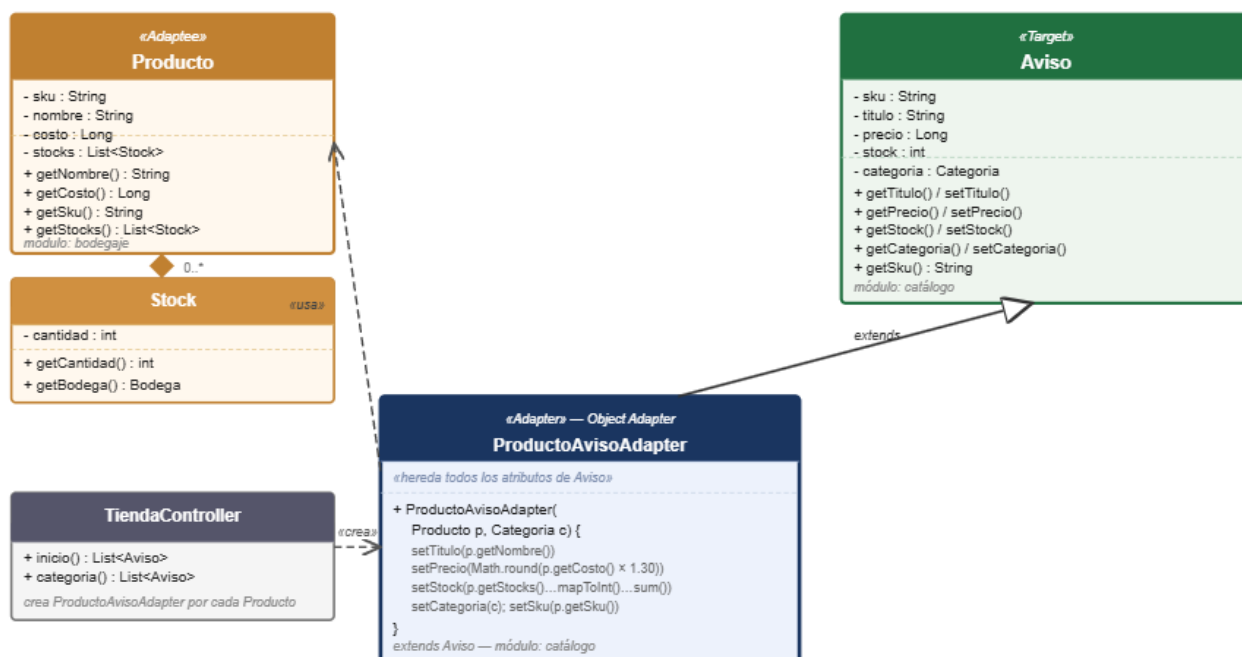
El patrón seleccionado es el **Composite** (GoF Estructural). Este patrón permite tratar objetos individuales y composiciones de objetos de manera uniforme, definiendo una interfaz común que implementan tanto los nodos hoja (*Leaf*) como los nodos compuestos (*Composite*).

Justificación de la elección:

- La interfaz ItemMenu ya provista por el diseñador define el método getHijos(), que es exactamente la operación característica del componente en el patrón Composite.
- La jerarquía de categorías es una estructura arbórea de profundidad variable: Composite es el único patrón del catálogo GoF diseñado específicamente para estructuras arbóreas recursivas con tratamiento uniforme de hojas y ramas.
- Un patrón Strategy no resuelve el problema de la recursividad. Un Decorator mantiene la misma interfaz, pero no agrega estructura de árbol. Un Factory crea objetos, pero no resuelve la necesidad de iteración recursiva uniforme.

Actividad 2 – Integración Bodegaje–Catálogo Conclusiones

Diagrama UML



Implementación del Código

Clase del módulo *catálogo*, usada directamente por las plantillas Thymeleaf. **No se modifica**: el patrón Adapter garantiza que el target permanezca intacto.

```

cl/patrones/taller/u2/catalogo/domain/Aviso.java
public class Aviso {
    private String sku, titulo, imagen;
    private Long precio;
    private int stock;
    private Categoria categoria;
}
  
```

Clase del módulo *bodegaje*. Contiene el costo real del artículo y la lista de stocks por bodega. **No se modifica**.

```

cl/patrones/taller/u2/bodegaje/domain/Producto.java
public class Producto {
    private String sku, nombre, imagen;
    private Long costo;
    private List<Stock> stocks = new ArrayList<>();
}
  
```

Clase núcleo del patrón. **Hereda de Aviso** (para ser tratada polimórficamente como Aviso por el cliente) y **recibe un Producto por constructor** (composición con el adaptee). El constructor ejecuta toda la traducción de una vez, por lo que el objeto resultante es inmutable desde la perspectiva del cliente.

```

cl/patrones/taller/u2/catalogo/domain/ProductoAvisoAdapter.java
public class ProductoAvisoAdapter extends Aviso {

    public ProductoAvisoAdapter(Producto producto, Categoria categoria) {
        setTitulo(producto.getNombre());
        setSku(producto.getSku());
        setImagen(producto.getImagen());
    }
}
  
```



```

        setCategoria(categoria);
        setPrecio(Math.round(producto.getCosto() * 1.30));
        setStock(producto.getStocks()
            .stream()
            .mapToInt(s -> s.getCantidad())
            .sum());
    }
}

```

El controlador opera exclusivamente sobre Aviso. La obtención de la Categoría asociada al SKU se realiza mediante `ClasificacionRepository.findFirstBySku()`, y el resultado se pasa al constructor del Adapter. El controlador nunca accede a propiedades de Producto directamente.

cl/patrones/taller/u2/tienda/controller/TiendaController.java

```

@GetMapping("/")
public String inicio(Model model) {

    List<Aviso> avisos = bodegaService.getProductos().stream()
        .map(producto -> {
            Categoria categoria = clasificacionRepository
                .findFirstBySku(producto.getSku())
                .map(c -> c.getCategoria())
                .orElse(null);
            return new ProductoAvisoAdapter(producto, categoria);
        })
        .collect(Collectors.toList());

    model.addAttribute("avisos", avisos);
    return "inicio";
}

```

Patrón Adapter Aplicado

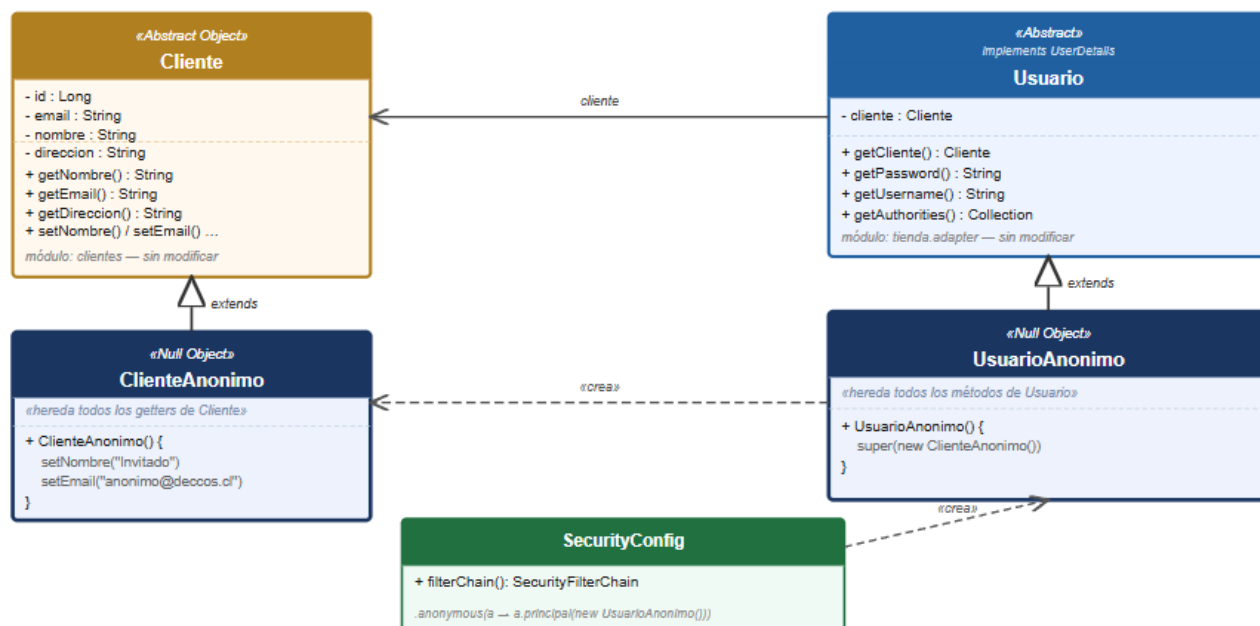
El patrón seleccionado es el **Adapter** en su variante Object Adapter (GoF Estructural). Hereda del target (Aviso) y recibe el adaptee (Producto) por constructor, traduciendo sus datos en el momento de la instanciación.

Justificación:

- **No es Facade:** Facade simplifica una interfaz compleja; aquí el problema no es complejidad sino incompatibilidad de tipos.
- **No es Decorator:** Decorator añade responsabilidades manteniendo la misma interfaz. Aquí las interfaces son distintas y se requiere presentar Producto como Aviso.
- **No es Builder:** Builder construye objetos complejos paso a paso; no traduce entre interfaces incompatibles.
- **Adapter es la solución canónica** cuando dos módulos pre-existentes deben colaborar sin modificarse, a través de una traducción de interfaz.

Actividad 3 – Usuario Anónimo

Diagrama UML



Implementación del Código

La clase base define la interfaz que el sistema espera. Tanto el cliente autenticado (cargado desde la BD) como el anónimo son instancias de esta clase. **No se modifica.**

cl/patrones/taller/u2/clientes/Cliente.java

```
public class Cliente {
    private String nombre, email, direccion, comuna;
}
```

Extiende Cliente e inicializa todos los campos con valores seguros y significativos. Cualquier llamada a getNombre(), getEmail(), etc. retorna un valor no nulo. No es necesario ningún método adicional: hereda la interfaz completa de Cliente.

cl/patrones/taller/u2/clientes/ClienteAnonimo.java

```
public class ClienteAnonimo extends Cliente {

    public ClienteAnonimo() {
        setNombre("Invitado");
        setEmail("anonimo@deccos.cl");
        setDireccion("");
        setComuna("");
    }
}
```

Implementa UserDetails de Spring Security envolviendo un Cliente. El método getCliente() expone el dominio al template Thymeleaf. No se modifica: el patrón Null Object actúa sobre el objeto que se le pasa, no sobre esta clase.

cl/patrones/taller/u2/tienda/adapter/Usuario.java

```
public class Usuario implements UserDetails {

    private Cliente cliente;
```

```

public Usuario(Cliente cliente) {
    this.cliente = cliente;
}

public Cliente getCliente() { return cliente; }

@Override
public String getPassword() { return cliente.getContrasena(); }

@Override
public String getUsername() { return cliente.getEmail(); }
}

```

Extiende Usuario pasando un ClienteAnonimo al constructor padre. Cuando Spring Security asigna este objeto como principal, la cadena principal.cliente.nombre resuelve correctamente a "Invitado" sin excepciones.

```

cl/patrones/taller/u2/tienda/adapters/UsuarioAnonimo.java
public class UsuarioAnonimo extends Usuario {

    public UsuarioAnonimo() {
        super(new ClienteAnonimo());
    }

}

```

El método .anonymous() del DSL de Spring Security permite reemplazar el principal anónimo por defecto (null) con cualquier objeto. Aquí se registra new UsuarioAnonimo(), de modo que toda request sin sesión recibe este objeto como #authentication.principal.

```

cl/patrones/taller/u2/SecurityConfig.java
@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Bean
    SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http
            .authorizeHttpRequests(authz -> authz.anyRequest().permitAll())
            .formLogin(form -> form
                .loginPage("/ingresar")
                .loginProcessingUrl("/ingresar")
                .permitAll())
            .logout(logout -> logout
                .logoutRequestMatcher(new AntPathRequestMatcher("/logout",
"GET")))
                .permitAll())
            .anonymous(anonymous -> anonymous.principal(new UsuarioAnonimo()));

        return http.build();
    }
}

```

Patrón Null Object Aplicado

El patrón seleccionado es el **Null Object** (Woolf, 1996). Este patrón reemplaza el uso de null por un objeto concreto que implementa la misma interfaz que el objeto real, pero con comportamiento neutro o por defecto. Elimina la necesidad de verificar nulos en el código cliente.

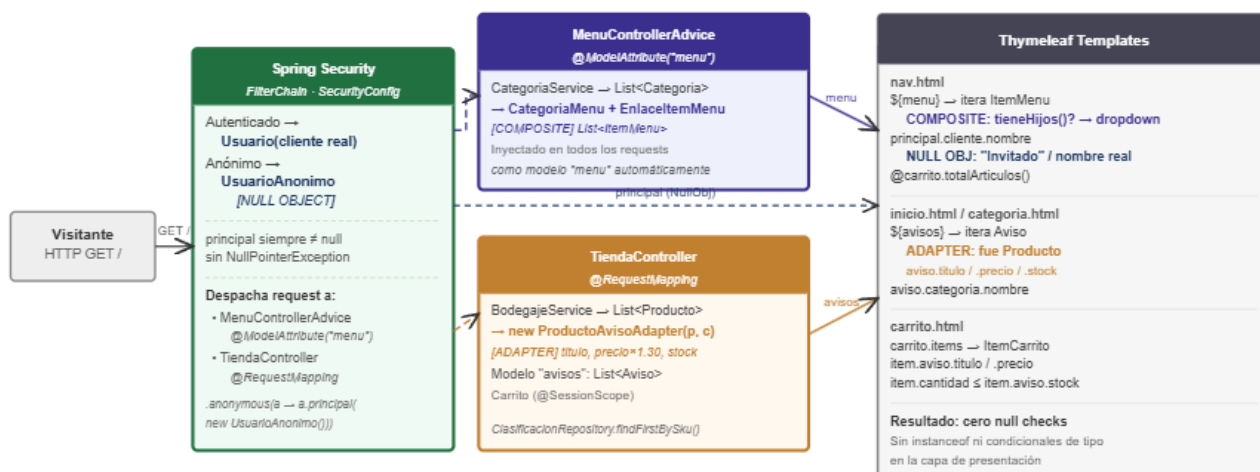
Justificación:

- **No es Strategy:** Strategy intercambia algoritmos; aquí no hay variación de comportamiento, solo se necesita un objeto seguro cuando no existe usuario.
- **No es Proxy:** Proxy controla acceso a un objeto existente. En este caso no hay usuario real al cual hacer proxy.
- **No es Factory:** Factory crea objetos pero no resuelve el problema de ausencia de un objeto en tiempo de ejecución.
- **Null Object** es la solución específica para eliminar verificaciones de nulo en cadena, proveer comportamiento por defecto y simplificar el código cliente sin condicionales adicionales.

Composición de Patrones y Desacoplamiento del Sistema

Los tres patrones implementados no son soluciones aisladas: colaboran en tiempo de ejecución durante cada ciclo de request-response de la aplicación. La composición de patrones resulta en un sistema donde cada módulo puede evolucionar de forma independiente.

Diagrama de Colaboración



Beneficios de la Composición

- **Módulos independientes:** *bodega*, *catálogo* y *clientes* no se conocen entre sí. Solo los patrones (ProductoAvisoAdapter, ClienteAnonimo) importan clases de más de un módulo.
- **Plantilla sin lógica de negocio:** nav.html y inicio.html no tienen condicionales de tipo, nulos ni conversiones. Todo el conocimiento de integración está en las clases de patrón.
- **Extensibilidad:** agregar un nuevo tipo de ítem de menú solo requiere crear una clase que implemente ItemMenu. Agregar una nueva fuente de productos solo requiere crear un nuevo Adapter. Agregar roles (invitado premium, etc.) solo requiere crear nuevas subclases de Cliente y Usuario.

Conclusión

El desarrollo del taller permitió aplicar tres patrones de diseño:

El patrón **Composite** solucionó el problema de representar una jerarquía arbórea de menú de manera uniforme para la plantilla Thymeleaf, permitiendo que tanto enlaces simples como categorías con subcategorías sean tratados por el mismo código de renderizado.

El patrón **Adapter** desacoplaron los módulos bodegaje y catálogo, que tienen modelos de dominio incompatibles. La traducción de campos (nombre -> título, costo $\times$ 1.30 -> precio, suma de stocks -> stock consolidado) queda encapsulada en un único constructor, visible y testeable.

El patrón **Null Object** eliminó los errores de NullPointerException al proporcionar objetos de dominio con comportamiento neutro ("Invitado") cuando el usuario no está autenticado. La integración con Spring Security se realizó sin modificar ninguna de las clases existentes.

La composición de los tres patrones produce un sistema donde los módulos de negocio son independientes, las plantillas no tienen lógica de integración, y el código es extensible sin modificar las clases base, esto optimizan el desacoplamiento y mejoran la mantenibilidad del sistema a largo plazo.